

# RESUMEN GENERAL DEL SISTEMA SISCO-EDU

## Sistema Distribuido de Asistencia Biométrica con ESP32 + AS608 + LoRa

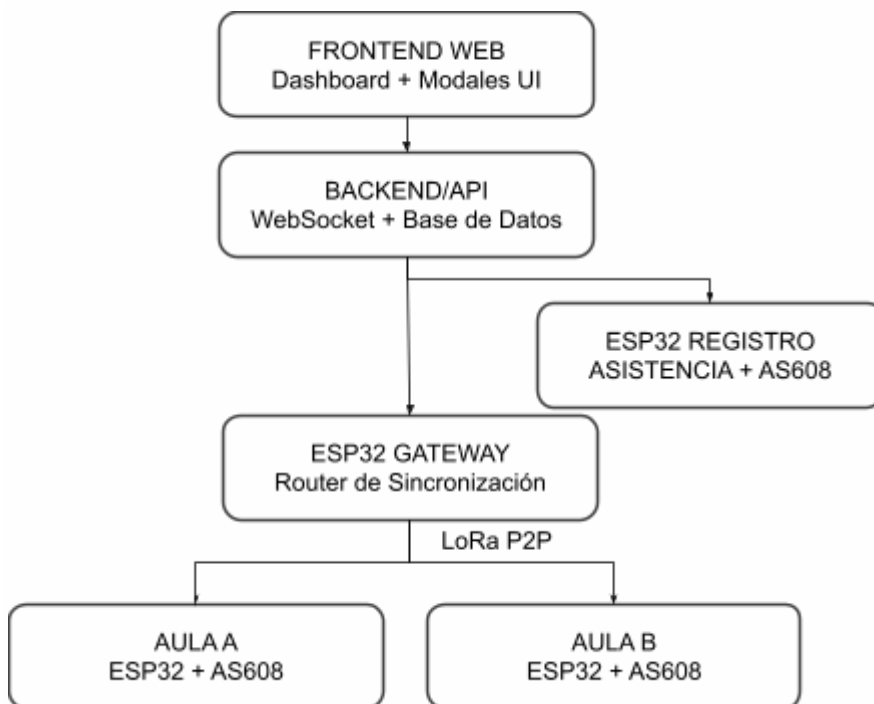
---

### OBJETIVO DEL PROYECTO

Construir un sistema de asistencia biométrica distribuido donde:

- Un frontend web administra estudiantes.
  - Un ESP32 biométrico capta huellas.
  - Las huellas se almacenan en una base de datos.
  - Un gateway distribuye templates biométricos.
  - Cada aula tiene un ESP32 + AS608 que valida localmente las huellas.
  - Las asistencias se sincronizan con el backend.
- 

### ARQUITECTURA GENERAL



### COMPONENTES DEL SISTEMA

#### FRONTEND WEB

Responsable de:

- listar estudiantes
- editar usuarios
- abrir modales

- capturar huellas remotamente
- visualizar estados

## ◆ **BACKEND**

Responsable de:

- API REST/WebSocket
- base de datos
- sincronización biométrica
- control de aulas
- dashboard de coordinación

## ◆ **ESP32 BIOMÉTRICO DE CAPTURA**

Responsable de:

- capturar huellas
- exportar templates
- convertir a HEX/Base64
- enviar templates al backend

## ◆ **ESP32 GATEWAY**

Responsable de:

- recibir templates del backend
- fragmentar paquetes
- distribuir vía LoRa
- recibir eventos de asistencia

## ◆ **ESP32 DE AULA**

Responsable de:

- almacenar templates en AS608
- validar huellas localmente
- registrar asistencia
- enviar eventos al gateway

## ◆ **SENSOR AS608**

Responsable de:

- capturar huellas
- generar templates
- almacenar templates
- comparar huellas localmente

---

## FLUJO COMPLETO DEL SISTEMA

### FASE A — Gestión de estudiantes

#### Objetivo

Crear o editar estudiantes desde el frontend.

#### Flujo

1. Usuario abre lista de estudiantes
2. Usuario selecciona:
  - crear
  - o
  - editar estudiante
3. Frontend abre modal:  
"Editar estudiante"

### Modal de estudiante

Contiene:

- nombre
- documento
- curso
- aula
- horario
- fingerprint\_data\_user

y un botón:

[ Capturar huella ]

### FASE B — Captura biométrica remota

#### Objetivo

Capturar huella desde un ESP32 remoto.

#### Arquitectura

Frontend

↓ WebSocket/API

Backend

↓ WiFi

ESP32 Biométrico

↓ UART

AS608

### Flujo

1. Usuario pulsa:  
[ Capturar huella ]
2. Frontend abre modal captura
3. Frontend envía:  
start\_capture
4. Backend retransmite al ESP32
5. ESP32 activa AS608
6. Usuario coloca dedo
7. AS608 genera template
8. ESP32 descarga template
9. ESP32 convierte template:  
binario → HEX/Base64
10. ESP32 envía template al backend
11. Backend retransmite al frontend
12. Frontend autocompleta:  
fingerprint\_data\_user



### IMPORTANTE

NO copiar/pegar manualmente el HEX.

Debe insertarse automáticamente.

---



## FASE C – Guardado del estudiante

### Objetivo

Persistir datos del usuario.

### Flujo

1. Usuario guarda formulario
2. Backend almacena:
  - user\_id\_global
  - room\_id
  - fingerprint\_data\_user
3. Backend marca:  
pendiente\_sync = true

---

## ◆ FASE D – Sincronización biométrica

### Objetivo

Enviar templates al aula correcta.

### Arquitectura

Backend

↓

ESP32 Gateway

↓ LoRa P2P

ESP32 Aula

↓ UART

AS608 Aula

### Flujo

1. Gateway consulta pendientes
2. Backend responde:
  - user\_id\_global
  - room\_id
  - fingerprint\_data\_user
3. Gateway:
  - fragmenta paquetes
  - agrega checksum
4. Gateway transmite vía LoRa
5. ESP32 Aula:
  - recibe fragmentos
  - reconstruye template
6. ESP32 Aula:
  - valida checksum
7. ESP32 Aula:
  - convierte HEX/Base64 → binario
8. ESP32 Aula:
  - carga template al AS608
9. AS608:
  - almacena template

## **IMPORTANTE**

Usar:

 LoRa P2P

NO LoRaWAN clásico.

### **Mapeo biométrico**

Cada aula debe almacenar:

user\_id\_global ↔ sensor\_slot

### **Tabla recomendada**

user\_id\_global | room\_id | sensor\_slot

---

## **FASE E – Asistencia**

### **Objetivo**

Validar asistencia localmente.

### **Flujo**

1. Usuario coloca dedo
2. AS608 compara localmente
3. AS608 devuelve:  
    sensor\_slot
4. ESP32 Aula:  
    traduce slot → user\_id\_global
5. ESP32 Aula:  
    genera evento asistencia
6. ESP32 Aula:  
    guarda evento localmente
7. ESP32 Aula:  
    transmite evento al gateway
8. Gateway:  
    retransmite al backend
9. Backend:  
    actualiza dashboard

---

## Eventos recomendados

### Asistencia

```
{  
  "cmd": "attendance",  
  "user_id": 125,  
  "room": "AULA_A",  
  "timestamp": 1715400012  
}
```

---

### Heartbeat

```
{  
  "cmd": "heartbeat",  
  "room": "AULA_A",  
  "status": "online",  
  "templates": 45  
}
```

---

## FASE F – Dashboard

### Objetivo

Visualizar asistencia.

#### Backend calcula

PRESENTES:

usuarios con eventos

AUSENTES:

usuarios esperados sin eventos

### IMPORTANTE

NO enviar listas de ausentes desde ESP32.

El backend debe calcularlos.

## **SEGURIDAD**

### **Cifrado**

Usar:

- HTTPS
- WSS
- AES-128

### **Validaciones**

Verificar:

- checksum
- tamaño
- integridad del template

### **Autenticación de nodos**

Cada ESP32 debe tener:

- node\_id único
  - clave privada
- 

## **RESILIENCIA**

### **Buffer local**

Guardar eventos localmente:

- SPIFFS
- LittleFS

### **Reintentos automáticos**

Si falla LoRa:

- guardar evento
- reenviar luego

### **Operación offline**

Cada aula debe funcionar:

- sin internet
- sin backend
- sin gateway temporalmente

---

## TEST MÁS IMPORTANTE DEL PROYECTO

### TEST CRÍTICO

1. Registrar huella en Sensor A
2. Exportar template
3. Guardar HEX/Base64
4. Enviar a Sensor B
5. Importar template
6. Verificar:
  - ¿Sensor B reconoce correctamente?

### ESTE TEST DEFINE TODO EL PROYECTO

Si funciona:

- toda la arquitectura es viable.

Si no funciona:

- debes cambiar de sensor biométrico.

### TECNOLOGÍAS RECOMENDADAS

| Componente         | Tecnología        |
|--------------------|-------------------|
| Frontend           | React / Vue       |
| Backend            | Node.js / Laravel |
| Tiempo real        | WebSocket         |
| Base de datos      | PostgreSQL/MySQL  |
| Gateway            | ESP32             |
| Comunicación aulas | LoRa P2P          |
| Biometría          | AS608             |
| Persistencia ESP32 | SPIFFS/LittleFS   |

---

## CONCLUSIÓN FINAL

La arquitectura final del sistema es:

- ✓ distribuida
- ✓ escalable
- ✓ modular
- ✓ resiliente
- ✓ compatible con múltiples aulas
- ✓ con validación biométrica local
- ✓ sincronización centralizada
- ✓ baja latencia
- ✓ tolerancia a fallos

---

## ORDEN RECOMENDADO DE IMPLEMENTACIÓN

### ETAPA 1

- ✓ Probar AS608 localmente.

### ETAPA 2

- ✓ Exportar/importar templates entre sensores.

### ETAPA 3

- ✓ Crear ESP32 biométrico de captura.

### ETAPA 4

- ✓ Crear frontend de captura biométrica.

### ETAPA 5

- ✓ Crear backend y base de datos.

### ETAPA 6

- ✓ Crear gateway LoRa.

### ETAPA 7

- ✓ Sincronizar templates hacia aulas.

### ETAPA 8

- ✓ Implementar asistencia local.

### ETAPA 9

- ✓ Dashboard y reportes.

### ETAPA 10

- ✓ Seguridad, cifrado y tolerancia a fallos.